AutoML에서 Autoresearch로

DOE는 Research Agent의 Harness가 될 수 있는가?

AutoML, MLOps, Research Workflow, DOE-guided Agent

실험 결과 영역은 이후 채움

1. 왜 지금 이 주제인가?

- Coding Agent는 실제 개발 workflow 안에서 빠르게 발전하고 있다.
- Research automation도 문헌 정리, 코드 수정, 실험 실행, 보고서 작성까지 넓어지고 있다.
- 하지만 연구 실험은 여전히 ad hoc 반복으로 흐르기 쉽다.
- 이제 질문은 "agent가 실험을 돌릴 수 있는가?"보다 "무슨 harness가 agent를 더 신뢰 가능하게 만드는가?"이다.

지금 보이는 변화	아직 약한 부분
Coding Agent의 실제 배포	Research loop의 규율
실험 자동화 도구 증가	실험 설계 표준
end-to-end research demo 확산	robustness / reproducibility

2. AutoML이란 무엇인가?

- AutoML은 모델 개발 과정의 일부 탐색을 자동화한다.
- 대표 대상은 `model selection`, `hyperparameter tuning`, `pipeline search`다.
- 더 넓게는 `feature engineering`과 `NAS`까지 확장된다.
- 핵심은 비교적 주어진 `search space` 안에서 성능을 최적화하는 것이다.

```
Data → Search over models/pipelines → Evaluation → Best configuration
```

3. 대표적인 AutoML 방법

방법	강점	한계
Grid / Random Search	단순하고 강건함	비효율적일 수 있음
Bayesian Optimization	sample-efficient	구조적 edit 공간에는 약할 수 있음
Evolutionary Search	구조 탐색에 강함	비용이 큼
Multi-fidelity / Hyperband	예산 효율이 좋음	proxy bias 위험
NAS	architecture 탐색 가능	탐색 비용이 높음

방법론의 차이보다, 탐색 전략과 예산 관리가 중요하다는 점을 강조한다.

4. AutoML과 MLOps의 연결

- AutoML만으로는 충분하지 않고, 실행 인프라가 함께 있어야 한다.
- `experiment tracking`, `orchestration`, `reproducibility`가 탐색을 실제 시스템으로 만든다.
- `model registry`, `deployment`, `monitoring`이 탐색을 운영으로 이어 준다.
- `cost governance`도 중요하다. 탐색은 수학 문제이기 전에 시스템 workload이기 때문이다.

Search 층	운영 층
candidate generation	tracking
evaluation	orchestration
best-config selection	registry / deployment / monitoring

5. 대학원생의 연구 루프

1. 문헌 조사
2. 가설 수립
3. 실험 설계
4. 구현 및 실행
5. 분석
6. 토론 및 수정
7. 다음 실험

`Literature → Hypothesis → Design → Implement/Run → Analyze → Revise → Next Loop`

6. Autoresearch란 무엇인가?

- Autoresearch는 단순한 `hyperparameter search`를 넘는다.
- `code`, `module`, `pipeline structure`까지 수정할 수 있다.
- `tool use`, `memory`, `literature`, `iterative reasoning`을 함께 사용할 수 있다.
- 따라서 `search space`는 AutoML보다 넓고, 덜 정돈되어 있으며, 더 open-ended하다.

AutoML	Autoresearch
config / architecture 중심	hypothesis / code / module / pipeline
explicit objective 최적화	objective + reasoning + iteration

7. Autoresearch 현황

현재 공개 생태계는 이미 `단일 demo` 보다 `layered ecosystem` 쪽으로 분화되고 있다.

curated source	현재 신호	분류 방식
`alvinreal/awesome-autoresearch`	GitHub 약 `1.2k` stars	descendants / research-agent systems / hardware forks / domain adaptations / benchmarks
`handsome-rich/Awesome-Auto-Research-Tools`	GitHub `154` stars	end-to-end systems / deep research / experiment & code agents / skills & plugins / surveys

- 즉 현재의 흐름은 "하나의 강한 agent"보다 "생태계 + 실행 인프라 + benchmark + reusable skill" 쪽으로 이동 중이다.

8. 사용례와 발전 방향

층	대표 예시	지금 보이는 방향
End-to-End	`karpathy/autoresearch`, `microsoft/RD-Agent`, `SakanaAI/AI-Scientist`	topic → experiment → paper 통합
Deep Research	`GPT Researcher`, `Open Deep Research`, `PaperQA2`	literature synthesis와 citation 강화
Experiment & Code	`OpenHands`, `Aider`, `SWE-agent`, `AIDE`	Coding Agent가 research pipeline의 hands가 됨
Skills / Infra	`AI-Research-SKILLs`, `claude-scientific-skills`	reusable workflow와 domain skill 분화

- curated list는 또 다른 방향도 보여 준다: `autokernel` 같은 GPU kernel optimization, `autovoiceevals` 같은 voice agent hardening, `autoresearch-sudoku` 같은 solver optimization처럼 domain별 adaptation이 늘고 있다.
- benchmark 축도 강해지고 있다: `MLE-bench`, `MLAgentBench`, `MLR-Bench`.

Source: Awesome-Auto-Research-Tools, awesome-autoresearch, accessed 2026-04-07

9. AutoML vs. Autoresearch

항목	AutoML	Autoresearch
탐색 대상	config, architecture	hypothesis, code, module, pipeline, experiment program
평가	explicit objective	objective + reasoning + iteration
인간 역할	search space 설계	research program 설계
주요 위험	비효율적 탐색	metric hacking, incoherent search
필요한 인프라	experiment infra	experiment + memory + harness

10. Harness gap

Coding Agent 쪽에서 이미 흔한 것	Research Agent 쪽에서 아직 약한 것
`TDD`	hypothesis discipline
`CI`	factor definition
regression test	robustness check
lint / build check	reproducible iteration structure
deploy guardrail	experiment promotion rule

- 그래서 Research Agent는 점수가 좋아 보여도 그 탐색이 coherent했는지 설명하기 어렵다.

11. DOE를 Harness 후보로 보기

DOE primitive	Agent loop에서의 역할
screening	중요한 factor를 초기에 좁힘
factorial thinking	interaction을 구조적으로 확인
sequential refinement	유망 구간을 단계적으로 세밀화
robustness / confirmation	우연한 win과 재현 가능한 win을 분리

- 따라서 DOE는 "무엇을, 어떤 순서로, 어떤 조합으로 실험할지"에 대한 규율을 제공한다.

12. Agent loop에 무엇이 달라지는가?

단계	Vanilla Agent	DOE-guided Agent
시작	idea 생성	factor 정의
초반	code change 후 바로 run	screening으로 후보 압축
중반	gain이 나면 계속 추적	interaction check
후반	best-so-far 갱신 중심	refinement + robustness
종료	score가 높으면 채택	promote rule로 승격

13. 실험 설정

Variant	설명	비교 목적
Vanilla Agent	unconstrained iterative experiments	기본 baseline
DOE Agent	DOE 기반 planning과 staging	harness 효과 측정
DOE + X Agent	DOE + 추가 전략 모듈	harness와 추가 전략의 결합 효과

- 평가는 `best score` 만이 아니라 `cost`, `robustness`, `portfolio behavior` 까지 함께 본다.

14. 결과 요약

- Placeholder only: 실험 완료 후 채움

Variant	Best score	Avg improvement	Runs	Cost	Robustness pass
Vanilla	TBD	TBD	TBD	TBD	TBD
DOE	TBD	TBD	TBD	TBD	TBD
DOE + X	TBD	TBD	TBD	TBD	TBD

15. 성능 향상 추이

- Placeholder only: 실험 완료 후 채움
- `experiment budget` 또는 `run count` 대비 `best-so-far metric`
- 수렴 속도, plateau 형태, sample efficiency 비교

`x-axis = budget / run count`

`y-axis = best-so-far metric`

최종 점수보다 learning dynamics의 차이를 보여주는 그래프다.

16. 실험 포트폴리오 또는 실패 구조

- Placeholder only: 실험 완료 후 채움
- Option A: `Tic / Tac / To` 비율
- Option B: `invalid / no gain / unstable / robustness fail / promoted`
- Option C: `information gain vs. performance gain`

17. 왜 DOE-guided Agent는 다를 수 있는가?

- `search space`가 더 구조화된다.
- 중복되거나 정보량이 낮은 실험이 줄어든다.
- 상호작용을 우연히가 아니라 의도적으로 본다.
- opportunistic improvement와 robustness 확인을 분리할 수 있다.
- experiment history가 재사용 가능한 지식으로 축적된다.

18. 한계

- DOE가 `literature review`나 `hypothesis generation` 자체를 대체하진 못한다.
- open-ended research space는 깔끔한 factorization이 어렵다.
- large-scale training에서는 replication 비용이 크다.
- classical DOE는 discrete structural edit 공간에 그대로 맞지 않을 수 있다.
- evaluation harness가 약하면 DOE도 쉽게 흔들린다.

19. 결론

- AutoML에서 Autoresearch로 갈수록 `search space`는 넓어진다.
- search space가 넓어질수록 더 강한 `harness`가 필요하다.
- DOE는 Research Agent의 experimentation harness로 매우 유력한 후보다.

> 다음 단계는 더 좋은 agent 자체보다 더 좋은 harness이다.

20. References

구분	예시
curated landscape	`alvinreal/awesome-autoresearch`, `handsome-rich/Awesome-Auto-Research-Tools`
end-to-end systems	`karpathy/autoresearch`, `microsoft/RD-Agent`, `SakanaAI/AI-Scientist`
deep research	`assafelovic/gpt-researcher`, `Open Deep Research`, `PaperQA2`
experiment/code agents	`OpenHands`, `Aider`, `SWE-agent`, `AIDE`
evaluation	`openai/mle-bench`, `snap-stanford/MLAgentBench`, `chchenhui/mlrbench`